Week 10: SPI Protocol & DMA - Notes

Table of contents

1 SPI Protocol Fundamentals

1.1 Overview

SPI (Serial Peripheral Interface) is a synchronous, full-duplex, master/slave serial protocol developed by Motorola in the mid-1980s. Unlike I2C, SPI was never formally standardized; vendors implement small variations, but the core 4-wire model is universal.

It uses four signals (plus ground):

- **SCK** (Serial Clock) — driven by master, every edge shifts one bit
- **MOSI** (Master Out, Slave In) — master-to-slave data
- **MISO** (Master In, Slave Out) — slave-to-master data
- **CS** (Chip Select, also SS or NSS) — active-LOW per slave

1.2 Key Characteristics

Feature	Description
Wires	4 (SCK, MOSI, MISO, CS) + ground
Topology	Single master, multi-slave with separate CS per slave
Direction	Full-duplex (simultaneous TX and RX)
Driver	Push-pull (no pull-ups required)
Speed	1 MHz – 50 MHz typical (much faster than I2C)
Addressing	Physical (CS line), no protocol-level addressing
Acknowledgment	None — master assumes the slave is listening

1.3 Push-Pull vs Open-Drain

The fundamental hardware difference between SPI and I2C:

- **I2C (open-drain):** drivers can only pull LOW; HIGH is achieved by an external pull-up. Edge speed is limited by R × C.
- **SPI (push-pull):** drivers actively switch both rails. Edge speed is limited only by gate switching time.

Consequence: SPI is roughly 10–100× faster than I2C at the cost of one wire per slave (CS) and the inability to share the bus among multiple masters.

1.4 The Shift-Register Model

Both master and slave contain an N-bit shift register (typically 8 bits). MOSI connects the master's output bit to the slave's input bit; MISO connects the slave's output bit to the master's input bit. SCK clocks both.

After 8 SCK edges, the byte that started in the master is now in the slave, and the byte that started in the slave is now in the master. This is the meaning of "full-duplex": each transfer is an *exchange*, not a one-way send or receive.

Implication: to read N bytes from a slave you must clock N bytes out (typically dummy 0xFF). To write N bytes you must accept N bytes back (typically discard them).

1.5 SPI Modes (CPOL/CPHA)

The SPI specification leaves two clock parameters undefined:

- **CPOL** (Clock Polarity): SCK's idle level. 0 = LOW, 1 = HIGH.
- **CPHA** (Clock Phase): which SCK edge samples the data. 0 = first edge, 1 = second edge.

Four combinations give four modes:

Mode	CPOL	CPHA	SCK idle	Sample edge	Common use
0	0	0	LOW	rising	Most common (W25Q, MAX31855, many ADCs)
1	0	1	LOW	falling	Some audio codecs
2	1	0	HIGH	falling	Less common
3	1	1	HIGH	rising	W25Q (also supports), some Atmel parts

Mode mismatch produces data that looks shifted by one bit on the scope. The mode is fixed by the slave's datasheet; the master must match.

1.6 Timing Rules

1. **CS first:** assert CS LOW *before* the first SCK edge.
2. **CS last:** release CS HIGH *after* the last SCK edge of the transfer.
3. **No mid-transfer release:** if a multi-byte command is in progress, do not release CS between bytes — many slaves treat the rising edge of CS as "command ended."
4. **Mode match:** master CPOL/CPHA must equal slave CPOL/CPHA.
5. **Setup/hold:** consult slave datasheet for minimum setup and hold times around the sample edge.

1.7 SPI vs I2C: Selection Criteria

Criterion	Pick I2C	Pick SPI
Many cheap sensors		
GPIO-constrained		
Multi-master needed		
High throughput needed (>1 Mbps)		
Single fast peripheral (display, flash, SD)		
Need full-duplex		
Long PCB traces	(if slow)	(if short)

2 SPI Hardware Design

2.1 Multi-Slave Topology

Multiple slaves share SCK, MOSI, and MISO; each has its own CS line driven by a unique GPIO on the master. Only the slave whose CS is LOW responds; all others tri-state their MISO output.

Cost: one extra GPIO per slave.

Benefit: no addressing overhead, no bus contention, instant slave selection.

2.2 Signal Integrity at High Speed

At low speeds (1 MHz), SPI is forgiving. As speed climbs above ~10 MHz, problems appear:

- **Crosstalk** between MOSI and MISO on parallel ribbon cables
- **Reflection** from impedance mismatches at trace stubs
- **Ground bounce** when the return path is shared with high-current loads

- **Capacitive loading** on long traces slows edges and creates undershoot

Mitigations:

- Keep wires short (< 10 cm for breadboard; < 5 cm for production at 25 MHz+)
- Use a continuous ground return next to each signal
- Add 22 Ω – 47 Ω series termination at the driver output
- Reduce SCK speed before debugging code if a bus is misbehaving

2.3 Voltage-Level Compatibility

SPI is not inherently 3.3 V or 5 V — it follows the supply rails of the chips on the bus. **Mixing 5 V and 3.3 V devices on the same SPI bus requires level shifting** (a 74LVC245 or similar). The W25Q used in the lab is 3.3 V only and will be damaged by 5 V signals.

3 Why DMA

3.1 The CPU Bandwidth Problem

A polled or interrupt-driven SPI transfer requires the CPU to:

1. Wait for "TX register empty" event
2. Write next byte to SPI->DR
3. Wait for "RX register full" event
4. Read SPI->DR
5. Repeat per byte

At 25 MHz SPI (~3 MB/s) on an 84 MHz Cortex-M4, this consumes essentially the entire CPU. Even at lower SPI speeds the per-byte interrupt overhead (~30–50 cycles for the interrupt entry alone) dominates.

The CPU cannot service a fast peripheral byte-by-byte and also do real work.

3.2 What DMA Provides

DMA (Direct Memory Access) is a hardware engine that transfers data between memory and peripherals (or memory and memory) without CPU involvement. The CPU configures the transfer once (source, destination, length, mode) and starts it; the DMA controller handles every byte; the CPU is interrupted only at the start and end.

For an N-byte transfer, the CPU does ~5 instructions instead of ~30N.

3.3 Polled vs IRQ vs DMA

Mode	API	CPU cost per byte	Use when
Polled (blocking)	`HAL_SPI_Transmit`	High; blocks until done	Initialization, very small transfers
IRQ (non-blocking)	`HAL_SPI_Transmit_IT`	One ISR per byte	Medium transfers, low-speed bus
DMA	`HAL_SPI_Transmit_DMA`	Two ISRs total	Large transfers, fast bus, streaming

3.4 DMA Concepts

- **Channel / Stream:** the DMA controller has multiple independent state machines. STM32F4 has 2 controllers × 8 streams.
- **Request:** a peripheral asserts a request line when it needs the DMA to move a byte.
- **Direction:** memory → peripheral (TX), peripheral → memory (RX), or memory → memory.
- **Increment:** whether the source/destination address increments after each transfer. Memory: yes (walk through buffer). Peripheral: no (single register).
- **Mode:** Normal (one-shot) or Circular (wraps to start, runs forever).
- **HT/TC interrupts:** Half-Transfer interrupt fires at the midpoint; Transfer-Complete fires at the end.

3.5 Circular DMA and Double Buffering

Circular mode is the foundation of all streaming applications (audio, ADC, continuous SPI):

1. Configure DMA in circular mode with a buffer of size 2N
2. DMA writes byte 0..N-1 → asserts HT interrupt
3. CPU processes byte 0..N-1 while DMA writes byte N..2N-1
4. DMA wraps to byte 0 → asserts TC interrupt
5. CPU processes byte N..2N-1 while DMA writes byte 0..N-1 (again)
6. Repeat forever

This is the **ping-pong** or **double-buffer** pattern. The CPU's processing rate must keep up with the *average* data rate, not the *instantaneous* one.

3.6 Common DMA Pitfalls

Pitfall	Symptom	Fix
Buffer on stack	Random data corruption	Use `static` or global buffers
NVIC not enabled	Callback never fires	Enable global interrupt for the SPI in NVIC tab
Wrong stream/channel	DMA does nothing	Cross-check the reference manual table
Memory-increment OFF	All-same-byte buffer	Set memory increment to YES
Peripheral-increment ON	Hard fault or garbage	Set peripheral increment to NO
Buffer not cache-flushed (M7)	Stale data	Use `SCB_CleanDCache_by_Addr` (not relevant on M4)

4 W25Q SPI Flash

4.1 Why W25Q

The Winbond W25Q-series is the de-facto SPI flash family for hobby and education. Available 8 Mbit – 256 Mbit, ~$1, mode 0 or mode 3, up to 104 MHz (we use 1–10 MHz in lab). Standard JEDEC command set means the same code works across capacities and across most competitor parts (Macronix, GigaDevice, Adesto).

In this course we use it as the **fault-event log** in W12, so the W10 lab establishes the I/O patterns we'll reuse later.

4.2 Memory Hierarchy

Level	Size	Notes
Chip	1 MB – 32 MB	Whole device
Block	64 KB	Largest erase unit
Block (small)	32 KB	Alternate erase unit
Sector	**4 KB**	**Smallest erase unit**
Page	**256 B**	**Largest program unit**

Critical rule: flash bits can be programmed ($1 \rightarrow 0$) one byte at a time within a page, but can only be erased ($0 \rightarrow 1$) one *sector* at a time. To rewrite a single byte you must erase a 4 KB sector and reprogram everything you wanted to keep. This drives the design of every flash file system.

4.3 Standard JEDEC Commands

Command	Hex	Effect
JEDEC ID	0x9F	Returns 3-byte manufacturer + device ID
Read Status Register 1	0x05	Returns status byte (busy bit at LSB)
Write Enable	0x06	Sets the WEL bit; required before any write/erase
Write Disable	0x04	Clears WEL
Read Data	0x03	Reads N bytes starting at 24-bit address
Page Program	0x02	Writes up to 256 bytes (within one page)
Sector Erase (4 KB)	0x20	Erases 4 KB sector containing the address
Block Erase (64 KB)	0xD8	Erases 64 KB block containing the address
Chip Erase	0xC7	Erases entire chip

4.4 Standard Operation Patterns

Read N bytes from address ADDR:

1. Assert CS LOW
2. Send 0x03, ADDR[23:16], ADDR[15:8], ADDR[7:0]
3. Receive N bytes (clock out 0xFF for each)
4. Release CS HIGH

Program N bytes ($\leq$ 256, within one page):

1. Assert CS LOW, send 0x06, release CS HIGH (Write Enable)
2. Assert CS LOW, send 0x02 + 24-bit address + N data bytes, release CS HIGH
3. Poll status register (0x05) until busy bit clears (~1 ms typical)

Erase 4 KB sector containing ADDR:

1. Assert CS LOW, send 0x06, release CS HIGH (Write Enable)
2. Assert CS LOW, send 0x20 + 24-bit address, release CS HIGH
3. Poll status register until busy bit clears (~50 ms typical)

4.5 JEDEC ID as Bring-up Sanity Check

The JEDEC ID command (0x9F) is the SPI equivalent of the I2C WHO_AM_I check from W08. It is the first thing to run after wiring is complete:

- Returns 0xEF, ID[1], ID[2] for Winbond parts
- 0xEF = Winbond manufacturer ID
- ID[1] and ID[2] encode capacity (e.g., W25Q16: 0x40 0x15)

If JEDEC ID returns the right bytes, the bus, mode, CS timing, and clock are all confirmed working. If it returns 0x00 or 0xFF, no further code is worth writing until the wiring is fixed.

5 Debugging SPI

5.1 The 4-Step Bring-Up Checklist

Before writing any "real" code:

1. **Power & ground continuity** — 3.3 V on chip's VCC pin, ground continuous to MCU
2. **CS idle HIGH** — scope the CS pin; should sit at 3.3 V when no transfer is in progress
3. **SCK silent at idle** — SCK only toggles inside CS-LOW windows
4. **JEDEC ID returns 0xEF** — known-good handshake confirms full bus health

5.2 Symptom Catalog

Symptom	Cause	Fix
All-0xFF responses	MISO not connected; or slave not selected	Check CS wiring; check CS asserted LOW during read
All-0x00 responses	MISO shorted to GND; chip unpowered	Check VCC; remove short
First RX byte garbage, rest correct	Slave not stable when CS dropped	Discard rx[0]; many drivers do this automatically
Data shifted by one bit	Wrong CPHA	Swap CPHA setting in CubeMX

Symptom	Cause	Fix
Works at 1 MHz, fails at 10 MHz	Signal integrity	Shorten wires; add ground return; reduce speed
Write reports success, read returns old data	Forgot Write Enable; or read before busy clear	Add 0x06 + poll busy
Erase reports success, data still 0x55	Wrong sector address; or timed read while erasing	Verify address bits; poll busy
Random hard fault during DMA	Buffer on stack; out-of-bounds DMA	Use static/global; check NDTR / size

5.3 Oscilloscope and Logic Analyzer Tips

For a 4-channel scope:

- Ch1: SCK, Ch2: MOSI, Ch3: MISO, Ch4: CS
- Trigger on CS falling edge
- Time base: ~5 µs/div for 1 MHz SPI; ~500 ns/div for 10 MHz

For a logic analyzer (Saleae, USB cheap clone):

- Same channel mapping
- Use the protocol decoder (most LAs ship with SPI decoder)
- Configure CPOL/CPHA and bit order to match your master
- Verify the JEDEC ID command (0x9F) returns 0xEF as the first byte

6 Quick Reference

6.1 STM32 HAL SPI API (polled)

```
HAL_SPI_Transmit(&hspi1, tx_buf, len, HAL_MAX_DELAY);
HAL_SPI_Receive(&hspi1, rx_buf, len, HAL_MAX_DELAY);
HAL_SPI_TransmitReceive(&hspi1, tx_buf, rx_buf, len, HAL_MAX_DELAY);
```

6.2 STM32 HAL SPI API (DMA)

```
HAL_SPI_TransmitReceive_DMA(&hspi1, tx_buf, rx_buf, len);
// Returns immediately; HAL_SPI_TxRxCpltCallback() fires on TC
```

Override the callback in your code:

```
void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi) {
    if (hspi->Instance == SPI1) {
        spi_busy = 0;   // signal main loop
    }
}
```

6.3 CS Helper Macro

```
#define CS_LOW()  HAL_GPIO_WritePin(CS_PORT, CS_PIN, GPIO_PIN_RESET)
#define CS_HIGH() HAL_GPIO_WritePin(CS_PORT, CS_PIN, GPIO_PIN_SET)
```

6.4 JEDEC ID Read

```
uint8_t tx[4] = { 0x9F, 0xFF, 0xFF, 0xFF };
uint8_t rx[4];
CS_LOW();
HAL_SPI_TransmitReceive(&hspi1, tx, rx, 4, HAL_MAX_DELAY);
CS_HIGH();
// rx[0] = garbage, rx[1] = manufacturer (expect 0xEF), rx[2..3] = device ID
```

6.5 SPI Mode Selection in CubeMX

- "Clock Polarity (CPOL)" dropdown: Low (CPOL=0) or High (CPOL=1)
- "Clock Phase (CPHA)" dropdown: 1 Edge (CPHA=0) or 2 Edge (CPHA=1)
- Match these to the slave datasheet's required mode

6.6 Speed Selection

```
Prescaler = APB clock / desired SPI clock
```

For STM32F401RE with APB2 = 84 MHz:

Prescaler	SCK frequency
2	42 MHz (too fast for breadboard)
16	5.25 MHz (good lab speed)
32	2.625 MHz
64	1.3 MHz (safe debugging speed)
256	328 kHz (logic-analyzer friendly)

7 Further Reading

- Winbond W25Q16JV datasheet (the chip in the kit)
- ST AN3109: "How to use STM32 SPI peripheral with DMA"
- ST RM0383 (STM32F401 Reference Manual), SPI and DMA chapters
- Sparkfun "Serial Peripheral Interface (SPI)" tutorial — good pictures of mode timing